TECHNICAL SPEC

06

Email-to-Staging Automation

Update Workflow Architecture

PREPARED FOR	PREPARED BY	DATE	DOCUMENT
Precision Structures Inc.	EBYG Automation — SSC Digital Services	March 19, 2026	06 of 17

Design Specification — Precision Structures Inc.

Version: 1.0 **Date:** March 18, 2026 **Status:** Draft **Site:** Static HTML/CSS/JS on Azure Static Web Apps **Domain:** precisionstructures.net

---

1. Overview

This document specifies a system that allows the Precision Structures team to request website updates via email. An AI layer parses the request into a structured change, which enters a review queue before any changes reach staging or production.

Core principle: AI assists with understanding and categorizing requests. Humans review, implement, and approve all changes.

---

2. System Architecture

```
                        EMAIL UPDATE WORKFLOW
                        =====================

┌────────────────┐
│  Client sends    │
│  email to        │
│  updates@        │
│  precision...net │
└────────┬────────┘
         │
         ▼
┌────────────────┐
│  Microsoft 365   │
│  Mailbox         │
│  (Shared inbox)  │
└────────┬────────┘
         │
         ▼
┌────────────────┐      ┌──────────────────┐
│  Power Automate  │──────▶│  Azure Function      │
│  Trigger:        │      │  (Node.js/Python)    │
│  New email       │      │                      │
│  received        │      │  1. Extract body/    │
└────────────────┘      │     attachments      │
                        │  2. Call Claude API  │
                        │  3. Parse into       │
                        │     structured       │
                        │     change request   │
                        └──────────┬──────────┘
                                   │
                  ┌────────────────┼────────────────┐
                  │                │                │
                  ▼                ▼                ▼
            ┌─────────┐      ┌─────────┐      ┌──────────┐
            │ CLEAR    │      │ AMBIGUOUS│      │ OUT OF SCOPE │
            │ Request  │      │ Request  │      │ (not a site  │
            │          │      │          │      │  update)     │
            └────┬────┘      └────┬────┘      └────┬────┘
                 │                 │                 │
                 ▼                 ▼                 ▼
            ┌─────────┐      ┌─────────┐      ┌──────────┐
            │ GitHub   │      │ Flag for │      │ Forward to   │
            │ Issue    │      │ human    │      │ general      │
            │ created  │      │ review   │      │ inbox        │
            └────┬────┘      └─────────┘      └──────────┘
                 │
                 ▼
            ┌──────────────────┐
            │  Developer reviews   │
            │  issue, implements   │
            │  on develop branch   │
            └──────────┬──────────┘
                       │
                       ▼
            ┌──────────────────┐
            │  staging (Azure SWA) │
```

```
└──────────┬──────────┘
           │
           ▼
┌──────────────────┐
│  Client notified     │
│  with staging link   │
│  to review changes   │
└──────────┬──────────┘
           │
           ▼
┌──────────────────┐
│  Client approves →   │
│  merge develop →     │
│  master → production │
└──────────────────┘
```

3. Workflow Steps — Detailed

Step 1: Email Submission

The client sends an email to a dedicated address (e.g., `updates@precisionstructures.net`).

Email contents may include:

- Plain text description of the desired change
- Attached images (new photos, updated logos, etc.)
- References to specific pages ("on the About page," "the gallery section")
- Multiple changes in a single email

Examples of valid requests:

- "Please update the phone number on the contact page to (801) 985-3000"
- "Replace the third photo in the gallery with the attached image"
- "Add a new testimonial from ABC Construction to the homepage"
- "Our hours are changing to 7am-5pm starting next month"

Step 2: Email Ingestion

Microsoft 365 / Power Automate:

- Shared mailbox monitored by Power Automate flow
- Trigger: new email arrives in the updates mailbox
- Flow extracts: sender, subject, body (plain text + HTML), attachments, timestamp
- Attachments saved to Azure Blob Storage with unique identifiers
- Payload sent to Azure Function via HTTP POST

Step 3: AI Parsing (Claude API)

The Azure Function calls the Claude API with a structured prompt:

AI is asked to produce:

```
{
  "request_id": "REQ-20260318-001",
  "received_at": "2026-03-18T14:30:00Z",
  "sender": "client@example.com",
  "confidence": "high | medium | low",
  "changes": [
    {
      "page": "contact.html",
      "section": "business-hours",
      "change_type": "text_update | image_replace | content_add | content_remove | layout_change",
      "description": "Update Monday-Friday hours from 8am-4pm to 7am-5pm",
      "current_content_summary": "Currently shows Mon-Fri 8am-4pm",
      "requested_content": "Mon-Fri 7am-5pm",
      "attachments": [],
      "ambiguities": []
    }
  ],
  "overall_notes": "Straightforward text update, single change.",
  "needs_clarification": false,
  "clarification_questions": []
}
```

AI confidence levels:

- **High:** Clear, specific request that maps to a known page/section
- **Medium:** Understandable intent but some ambiguity (e.g., "update the photos" without specifying which)
- **Low:** Vague or contradictory request; multiple valid interpretations

Step 4: Review Queue

Based on AI parsing results:

CONFIDENCE	ACTION
High	GitHub Issue created automatically with parsed details
Medium	GitHub Issue created, flagged for clarification, developer reviews before implementing
Low	Notification sent to developer; original email forwarded for manual review
Out of scope	Auto-reply suggesting the correct contact address

Step 5: Implementation

- Developer reviews the GitHub Issue

- Azure SWA automatically deploys to staging
- Developer comments on the issue with the staging preview URL

Step 6: Client Review and Approval

- Client receives email with staging link and summary of changes
- Client replies with approval or revision requests
- On approval: developer merges `develop` into `master`
- Production deployment is automatic via Azure SWA

---

4. Where AI Is Allowed vs. Not Allowed

AI IS allowed to:

- Parse email text to identify the intent of the change request
- Categorize the type of change (text update, image swap, new content, removal)
- Identify which page and section the change likely applies to
- Draft a human-readable description of the requested change
- Flag ambiguities and suggest clarification questions
- Summarize attachments (describe image content)
- Detect if a request is out of scope (not a website update)

AI is NOT allowed to:

- Modify any file in the repository directly
- Approve or reject change requests
- Publish changes to staging or production
- Handle security-sensitive changes (login pages, forms, API keys)
- Make assumptions about unstated requirements
- Auto-respond to the client on behalf of the team without review

---

5. Safety Requirements

Human-in-the-Loop (Mandatory)

Every change flows through a human reviewer before reaching any environment. There is no automated path from email to deployed code.

Content Safety

- AI checks for inappropriate or off-brand content in requests
- Attached images are scanned for basic safety (no explicit content)
- Changes are verified against the original request before deployment

Hallucination Prevention

- AI produces structured data, not code changes
- All AI output is reviewed by a human before action
- AI includes confidence scores so reviewers know when to be cautious
- AI cites the specific email text that led to each parsed change

Ambiguity Handling

- Requests with confidence below "high" are flagged
- Clarification questions are generated automatically
- No implementation begins until the request is unambiguous

Audit Trail

Every request tracks:

- **Who** requested the change (sender email, timestamp)
- **What** the AI parsed (full structured output, confidence)
- **Who** reviewed it (developer name, review timestamp)
- **What** was implemented (commit SHA, branch, files changed)
- **Who** approved it (client approval email, timestamp)
- **When** it went live (merge timestamp, deployment confirmation)

Audit records are stored as GitHub Issue threads (MVP) or in a dedicated database (V2+).

---

6. Security Considerations

Email Security

- Sender verification: only process emails from a pre-approved list of client addresses
- SPF/DKIM/DMARC validation on the mailbox

- Size limits on attachments (e.g., 10 MB per file, 25 MB total)

API Security

- Claude API key stored in Azure Key Vault, never in code
- Azure Function uses managed identity for resource access
- All HTTP endpoints require authentication
- Rate limiting on the Azure Function to prevent abuse

Data Handling

- Email contents processed in-memory, not stored beyond what's needed
- Attachments stored in private Azure Blob Storage with time-limited SAS tokens
- No client PII stored outside of the GitHub Issue (which is in a private repo)
- Audit logs retained for 12 months minimum

Access Control

- Only authorized team members can view the review queue
- Only authorized team members can merge to production
- GitHub branch protection rules enforce review requirements
- Azure SWA deployment tokens secured and rotated regularly

---

7. MVP vs. Full Version Comparison

MVP (Recommended Starting Point)

Timeline: 2-3 weeks **Cost:** Minimal (Power Automate included with M365, Azure Function free tier, Claude API usage-based)

COMPONENT	IMPLEMENTATION
Email ingestion	Microsoft 365 shared mailbox + Power Automate
AI parsing	Azure Function calling Claude API
Review queue	GitHub Issues with labels (priority, status, page)
Notifications	Email notifications via Power Automate
Implementation	Manual by developer
Client review	Staging URL sent via email
Approval	Client replies to email thread
Audit trail	GitHub Issue comment history

Limitations:

- No visual dashboard — review happens in GitHub
- Client needs to understand staging URLs
- No automated diff generation
- Manual tracking of request status

V2: Automation + Dashboard

Timeline: 6-8 weeks (after MVP is validated) **Adds:**

- Simple web dashboard for viewing and managing requests
- Status tracking: received → parsed → in review → implementing → staging → approved → live
- Automated staging preview link generation
- Client-facing approval page (click to approve/request changes)
- Email templates for client communication
- Basic analytics: requests per week, average turnaround time

V3: AI-Assisted Implementation

Timeline: 10-14 weeks (after V2 is stable) **Adds:**

- AI generates proposed HTML diffs for review
- Side-by-side preview: current vs. proposed
- One-click apply for AI-generated changes (after human review)
- Image optimization pipeline for uploaded photos
- Batch processing for multi-change requests
- CRM integration for tracking client communication

8. Technical Requirements

Infrastructure

- Microsoft 365 Business (existing)
- Azure subscription (existing — used for SWA)
- Azure Function App (Consumption plan, Node.js or Python)
- Azure Blob Storage (for attachments)
- Azure Key Vault (for API keys)
- Claude API account with appropriate usage tier

Email Setup

- Create shared mailbox: `updates@precisionstructures.net`
- Configure Power Automate flow with appropriate triggers and error handling
- Set up auto-reply for received emails ("We've received your update request...")

GitHub Configuration

- Issue templates for parsed change requests
- Labels: `website-update`, `needs-clarification`, `approved`, `in-progress`, `deployed`
- Branch protection on `master`: require PR review before merge

9. Error Handling

SCENARIO	RESPONSE
Email parsing fails	Original email forwarded to developer with error details
Claude API unavailable	Request queued for retry (3 attempts, 5-minute intervals), then manual fallback
Attachment too large	Auto-reply asking client to use a file-sharing link
Unrecognized sender	Email ignored; no auto-reply (prevents spam acknowledgment)
Multiple conflicting changes	Flag all as "needs clarification," list the conflicts
Request references a page that doesn't exist	AI flags as ambiguous, suggests closest matching page

10. Success Metrics

- **Request turnaround time:** Target < 24 hours for simple text changes
- **Parsing accuracy:** AI correctly identifies page/section > 90% of the time
- **Clarification rate:** < 20% of requests need clarification follow-up
- **Client satisfaction:** Changes match what was requested on first implementation > 85% of the time
- **Zero unreviewed deployments:** 100% of changes pass through human review